1 CS336 Assignment 1: Building a Transformer LM

Ruiyu Li | Spring 2025

1.1 Part 1: Byte-Pair Encoding Tokenizer

1.1.1 Problem unicode1: Understanding Unicode

(a) `chr(0)` returns the null character (U+0000), also known as the NUL control character.

(b) Its `repr()` displays as the escape sequence `'\x00'`, while its printed representation is invisible — `print(chr(0))` produces no visible output.

(c) The null character is invisible in printed text but still occupies a position in the string. In Python, it is treated as a regular (though non-printing) character; for example, `"hello" + chr(0) + "world"` has length 11 despite appearing as `helloworld` when printed.

1.1.2 Problem unicode2: Unicode Encodings

(a) UTF-8 is preferable because it represents ASCII characters (which dominate web text) using only 1 byte each, making it far more compact than UTF-16 (minimum 2 bytes) or UTF-32 (always 4 bytes). Additionally, the UTF-8 byte vocabulary has a fixed size of 256, which is manageable for tokenizer training, whereas UTF-16 or UTF-32 would require a much larger initial vocabulary.

(b) The function is incorrect because UTF-8 is a variable-length encoding where a single Unicode character may be represented by 2–4 bytes. Decoding each byte individually fails for any multi-byte character. For example, the Japanese character `"こ"` encodes to `[227, 129, 147]` in UTF-8; attempting to decode the first byte alone (`bytes([227]).decode("utf-8")`) raises a `UnicodeDecodeError` because `0xe3` is the start byte of a 3-byte sequence and cannot be decoded in isolation.

(c) The two-byte sequence `bytes([0xff, 0xfe])` does not decode to any Unicode character(s). The bytes `0xFF` and `0xFE` are never valid in UTF-8 (valid lead bytes are at most `0xF4`, and continuation bytes range from `0x80` to `0xBF`), so this sequence is unconditionally invalid UTF-8.

1.1.3 Problem train_bpe_tinystories

(a) Training the BPE tokenizer on TinyStories with a vocabulary size of 10,000 took approximately 11 minutes (662.8s) on CPU with no GPU required. Peak memory usage was well within 30GB. The longest token in the vocabulary is `accomplishment` (15 bytes including the leading space), which makes sense given TinyStories consists of children's stories where moral and achievement-related words appear frequently enough to be merged into a single token.

(b) Based on profiling, the pre-tokenization step (regex matching and frequency counting across the corpus) dominates training time, while the iterative BPE merge step is comparatively fast due to incremental pair count updates.

1.1.4 Problem train_bpe_expts_owt

(a) Training a BPE tokenizer on OpenWebText with vocabulary size 32,000 took approximately 14 hours on CPU. The longest token is a 64-byte sequence of repeated malformed UTF-8 encodings (`\xc3\x83\xc3\x82` repeated 16 times), which represents garbled text from web crawl noise — repeated encoding errors get merged together because they appear frequently enough in the corpus.

(b) Compared to the TinyStories tokenizer, the OWT tokenizer has a much larger vocabulary (32K vs 10K) and is trained on more diverse text. TinyStories tokens tend to be common English words and word-pieces from simple children's stories, while OWT tokens cover a broader vocabulary including technical terms, proper nouns, and web-specific patterns. The OWT tokenizer achieves better compression on web text (4.55 bytes/token) versus the TinyStories tokenizer applied to OWT data (3.17 bytes/token), but would perform poorly on domain-specific corpora far from web language.

1.1.5 Problem tokenizer_experiments

(a) Compression ratios using each tokenizer on its native domain:

- TinyStories tokenizer on TinyStories data: **4.10 bytes/token**
- OWT tokenizer on OWT data: **4.55 bytes/token**

The OWT tokenizer achieves higher compression because its larger vocabulary (32K vs 10K) allows longer, more frequent subword units to be represented as single tokens.

(b) Applying the TinyStories tokenizer to OWT data yields only 3.17 bytes/token — significantly worse than the OWT tokenizer's 4.55. This is because the TinyStories vocabulary is optimized for simple English children's stories and lacks tokens for the diverse vocabulary, technical terms, and web-specific patterns in OWT. Many OWT words must be split into more byte-level fragments, increasing sequence length and reducing compression.

(c) The tokenizer processes approximately 1.95 MB/s on CPU. At this throughput, tokenizing the Pile dataset (825 GB) would take approximately 117.5 hours (5 days). In practice, this can be parallelized across multiple CPU cores to reduce wall-clock time significantly.

(d) `uint16` is appropriate because our largest vocabulary size is 32,000 (for OpenWebText), which fits within `uint16`'s range of 0–65,535. Compared to `uint32`, `uint16` halves the storage cost of the tokenized dataset with no loss of information.

1.2 Part 2: Transformer Language Model Architecture

1.2.1 Problem transformer_accounting

(a) GPT-2 XL has approximately 2.13 billion trainable parameters:

- Token embedding: $50\{,\}257 \times 1\{,\}600 = 80.4\text{M}$
- Per Transformer block: $4 \times 1\{,\}600^2 + 3 \times 6\{,\}400 \times 1\{,\}600 + 2 \times 1\{,\}600 \approx 41.0\text{M}$, times 48 layers = 1,966M
- Final RMSNorm + LM head: 80.4M

Total $\approx$ 2.13B parameters. At float32 (4 bytes/parameter), loading this model requires approximately **8.52 GB** of memory.

(b) The dominant matrix multiplies per forward pass (seq_len = 1,024):

Per block: QKV projections (15.7 GFLOPs) + attention $QK^\top$ and AV (6.7 GFLOPs) + output projection (5.2 GFLOPs) + FFN w1/w2/w3 (62.9 GFLOPs) = 90.6 GFLOPs per block.

Total: $48 \times 90.6 + 164.7 \approx$ **4.51 TFLOPs**.

(c) The FFN sublayer dominates, accounting for 67% of total FLOPs (3,019 out of 4,513 GFLOPs). This is because $d_{\text{ff}} = 4 \times d_{\text{model}}$, making the three FFN weight matrices collectively much larger than the four attention projection matrices.

(d) As model size increases from Small to XL, the FFN sublayer takes up a proportionally larger share of FLOPs, while the lm_head contribution shrinks:

Model	Params	Attn %	FFN %	lm_head %
Small (12L, 768d)	124M	20%	71%	9%
Medium (24L, 1024d)	345M	19%	73%	8%
Large (36L, 1280d)	762M	18%	74%	8%
XL (48L, 1600d)	2.13B	17%	74%	7%

This trend occurs because FFN FLOPs scale as $O(L \cdot d_{\text{model}}^2)$ while lm_head scales as $O(L \cdot d_{\text{model}} \cdot \text{vocab\_size})$ — the former grows faster with d_{model}.

(e) Increasing context_length from 1,024 to 16,384 (16×) changes the FLOPs dramatically. Attention FLOPs scale as $O(L^2)$, so they increase by 256 ×, while FFN FLOPs scale as $O(L)$ and increase by only 16 ×. As a result, attention becomes the dominant cost, rising from 17% to 78% of total FLOPs. This is why efficient attention mechanisms (e.g., FlashAttention, linear attention) are critical for long-context models.

1.3 Part 3: Training

1.3.1 Problem learning_rate_tuning

Running the toy SGD example with learning rates 10, 100, and 1000 for 10 iterations:

- **lr = 10**: Loss decreases rapidly but with noticeable oscillation around the minimum.
- **lr = 100**: Loss diverges immediately — parameter updates are far too large.
- **lr = 1000**: Loss diverges catastrophically within the first step, reaching extremely large values.

This demonstrates that SGD is highly sensitive to learning rate: values even modestly above the stable range cause immediate divergence. The decaying schedule $\frac{\alpha}{\sqrt{t+1}}$ helps mitigate this by reducing step sizes over time, providing a form of implicit regularization.

1.3.2 Problem adamwAccounting

(a) Peak memory breakdown (float32, $d_{\text{ff}} = 4 \times d_{\text{model}}$):

- Parameters: $4N$ bytes
- Gradients: $4N$ bytes
- AdamW optimizer state (m and v): $8N$ bytes
- Activations per block: $8BL^2 + 58BLd$ bytes, times n layers
- Output logits: $4BL \cdot \text{vocab\_size}$ bytes

Total: $M = 16N + n(8BL^2 + 58BLd) + 4BL(d + \text{vocab\_size})$

(b) For GPT-2 XL, fixed costs (parameters + gradients + optimizer) total 34.1 GB. Each unit of batch size contributes 5.06 GB in activations. The expression is:

$$M \approx 34.1 + 5.06 \times B \quad \text{GB}$$

Maximum batch size within 80 GB: $\left\lfloor \frac{80-34.1}{5.06} \right\rfloor = 9$.

(c) One AdamW step requires approximately $3F + 10N$ FLOPs, where $F \approx 4.51$ TFLOPs is the forward pass cost and $10N$ accounts for the moment updates and parameter update (10 ops/ parameter). Total ≈ 13.55 TFLOPs/step.

(d) At 50% MFU on an A100 (9.75 TFLOP/s effective), each step takes 1.39 seconds. Training for 400K steps with batch size 1,024 would require approximately **6.4 days** on a single A100.

1.4 Part 4: Experiments

1.4.1 Problem experiment_log

All experiments were tracked by logging training loss, validation loss, learning rate, and wall-clock time to console at regular intervals (every 50–100 steps). Validation loss was evaluated every 500 steps on a fixed held-out set using 20 random batches. Checkpoints were saved every 1,000 steps. The following sections report results from all experimental runs. Key metrics tracked: training loss per step, validation loss per 500 steps, wall-clock time, and final validation loss at step 5,000.

1.4.2 Problem learning_rate

(a) Learning rate sweep results (cosine annealing schedule, 5,000 steps, batch size 32, context length 256):

LR	Val@500	Val@2500	Final Val
1e-4	3.29	2.43	2.18
3e-4	2.80	2.06	1.86
1e-3	**2.30**	**2.10**	**1.69**
3e-3	2.41	1.95	1.75
1e-1 (const, no warmup)	N/A	N/A	Diverges

The optimal learning rate is 1e-3, achieving a final validation loss of 1.69. Smaller learning rates (1e-4, 3e-4) converge too slowly and remain significantly above the optimum within the same compute budget. Larger rates (3e-3) also underperform, suggesting the optimizer overshoots sharp minima. Search strategy: use 1e-3 as anchor and sweep one order of magnitude in each direction.

(b) At lr=1e-1 (constant, no warmup), the training loss immediately spikes from 9.28 to 25.9 at step 5 and 34.4 at step 10 — a clear divergence signature. Gradient clipping partially stabilizes training, but the model remains stuck at loss ≈ 5.8. The divergence threshold (1e-1) is approximately 100× the optimal learning rate (1e-3), consistent with the empirical observation that "the best learning rate sits near the edge of stability."

1.4.3 Problem batch_size_experiment

Batch Size	Tokens Processed	Final Val Loss	Wall Time
1	1.28M	3.07	151s
32	40.96M	1.69	2,500s
128	163.84M	1.47	8,754s

Larger batch sizes achieve lower validation loss within the same number of gradient steps, but primarily because they process proportionally more tokens per step. When controlling for total tokens processed, batch size 32 and 128 perform comparably — the apparent advantage of larger batches disappears.

Batch size 1 performs poorly due to extremely noisy gradient estimates: a single example provides an unreliable signal, causing the training loss to fluctuate wildly (ranging from 2.2 to 4.5 within the same run) and preventing effective convergence.

The key takeaway is that batch size primarily trades off **wall-clock efficiency** (larger batches better utilize GPU parallelism) against **statistical efficiency** (smaller batches provide more gradient updates per token). For a fixed compute budget, moderate batch sizes (32–128) are preferable.

1.4.4 Problem generate

The following text was generated using temperature $\tau = 0.8$ and top-$p = 0.95$ sampling with the prompt "Once upon a time":

"Once upon a time, there was a little girl named Lily. She loved to drink water. One day, she saw a big, red apple on the table. She was very happy. Lily wanted to drink the apple, but it was too high for her. So, she sat down on the table and started to drink the juice. Suddenly, a big bird flew down and took the apple from Lily. Lily was sad and cried. The bird saw Lily was sad and wanted to help. It flew up and got some apples for Lily. Together, they drank the apples. They became good friends and played at the park every day."

The output is fluent and grammatically correct, following the typical TinyStories narrative structure with a clear arc. Two factors that affect output quality are:

1. **Temperature**: Lower temperature ($\tau \to 0$) produces more repetitive but grammatically safe text, while higher temperature increases diversity at the cost of coherence. Our setting of $\tau = 0.8$ strikes a balance.
2. **Model size and training tokens**: With only 22.7M parameters trained on 1.3B tokens, the model occasionally produces minor semantic inconsistencies (e.g., "drink the apple" instead of "eat the apple"), reflecting the limited world knowledge of a small model.

1.4.5 Problem layer_norm_ablation

Training without RMSNorm at the optimal learning rate (1e-3) remains stable throughout — no divergence is observed. However, the initial loss is significantly higher (16.03 vs 9.24), indicating that RMSNorm greatly improves optimization at initialization. By the end of training, the no-norm model achieves a validation loss of 1.72, only marginally worse than the baseline of 1.69.

This suggests that RMSNorm's primary benefit is stabilizing early training dynamics rather than enabling fundamentally better convergence. The pre-norm architecture provides a cleaner gradient path through the residual stream, which is especially important in the first few hundred steps when activations can be poorly scaled.

1.4.6 Problem pre_norm_ablation

Switching from pre-norm to post-norm Transformer blocks with the same learning rate (1e-3) trains stably and achieves a final validation loss of 1.68, marginally better than the pre-norm baseline of 1.69.

Unlike large-scale training where post-norm is known to cause instability or require careful learning rate warmup, at this small scale (22.7M parameters, 5K steps) both variants converge reliably. The pre-norm architecture is nevertheless preferred in practice because it provides more stable gradients at large scale — the residual stream remains unnormalized, giving gradients a direct path through the network that becomes increasingly important as depth increases.

1.4.7 Problem no_pos_emb

Removing RoPE (NoPE) leads to a consistently higher validation loss throughout training, with a final val_loss of 1.79 compared to 1.69 with RoPE — a gap of 0.10 nats. Convergence is also slower in the early stages (step 500 val_loss: 2.76 vs 2.30).

This confirms that while decoder-only Transformers with causal masking can in theory infer relative positions implicitly, explicit positional encoding via RoPE provides a meaningful benefit at this scale. RoPE encodes relative position directly into the attention scores via rotation, giving the model a stronger inductive bias about token order from the start of training.

1.4.8 Problem swiglu_ablation

Replacing SwiGLU with a standard SiLU feed-forward network ($\text{FFN}_{\text{SiLU}(x)} = W_2\ \text{SiLU}(W_1 x)$, $d_{\text{ff}} = 4 \times d_{\text{model}}$) increases the parameter count slightly from 22.7M to 22.8M while achieving a final validation loss of 1.75 vs 1.69 for SwiGLU — a degradation of 0.06 nats.

The gating mechanism in SwiGLU allows the network to selectively suppress or amplify information at each position, providing additional expressiveness over a plain activation function. This empirically confirms Shazeer (2020)'s finding that GLU variants consistently outperform non-gated counterparts on language modeling tasks.

1.4.9 Problem main_experiment

Training the same model architecture (4 layers, d_model=512, 22.7M non-embedding parameters) on OpenWebText for 5,000 steps yields a final validation loss of 4.22, compared to 1.69 on TinyStories.

The higher loss reflects the fundamentally greater complexity of web text: TinyStories consists of simple, repetitive children's stories with limited vocabulary and predictable structure, while OpenWebText covers diverse topics, writing styles, and vocabulary from across the internet.

The loss difference should be interpreted carefully — the two numbers are not directly comparable as measures of model quality, since they measure perplexity on distributions of very different entropy. A loss of 4.22 on OWT corresponds to a perplexity of $e^{4.22} \approx 68$, meaning the model assigns on average $\frac{1}{68}$ probability to the correct next token. With the same compute budget, better results would require either a larger model, more training steps, or longer context length.

The following text was generated from the OWT model with prompt "The researchers found that" (temperature=0.8, top-p=0.95):

"The researchers found that participants were not able to get the same results. "These findings contribute to a family experience, increasing the number of participants who didn't feel good at the time of their study," one researcher said. Concerns also used a positive to predict when learning whether patients will really have autism and how they can't understand their data. "Some of these findings are really relevant to their projections," said another general manager."

The output mimics the surface form of academic news writing — complete sentences, quoted sources, hedged language — but lacks semantic coherence. Two key factors explain the lower quality compared to TinyStories:

1. **Insufficient model capacity**: With only 45.2M parameters and 5,000 training steps, the model cannot capture the rich, diverse patterns in web text. TinyStories has far lower entropy (simple vocabulary, repetitive structure), making it much easier to model at this scale.
2. **Domain diversity**: OWT mixes news, blogs, forums, and technical writing, while TinyStories is a single coherent domain. The model must spread its capacity across many styles, learning each one shallowly.